

Lecture 14: Application of DFS and BFS

*Instructor: Sourav Chakraborty**Scribe: Ajaya Kumar Patel, Suman Singha*

Depth First Search (DFS)

DFS is a **graph traversal algorithm** that explores as far as possible along each branch before backtracking.

While running DFS we maintain some information for each vertex

Variable	Meaning
visited[u]	Whether vertex is visited
discovery time $d[u]$	Time when vertex is first visited
finishing time $f[u]$	Time when DFS finishes exploring the vertex

Algorithm 1 Iterative Depth First Search

Input: Graph $G(V, E)$ **Result:** DFS traversal

```

while there exists an unvisited vertex u do
    Push( $u$ ) make  $u$  visited
    while stack is not empty do
         $u \leftarrow$  TOS
        if  $u$  has an unvisited neighbor v then
            Push( $v$ ) make  $v$  visited
        end
        else
            Pop( $u$ ) make  $u$  visited
        end
    end
end

```

Algorithm 2 Recursive Depth First Search

Input: Graph $G(V, E)$, starting vertex u

Result: DFS traversal

Function $DFS(u)$:
 make u visited print u
 for *each neighbor v of u* **do**
 if v is not visited **then**
 DFS(v)
 end
 end

Algorithm	Time Complexity
Iterative DFS	$O(V + E)$
Recursive DFS	$O(V + E)$

Table 1: Time Complexity of DFS Algorithms

Equivalence Relation in Graphs

Define a relation R on vertices such that uRv if there is a path between u and v . This relation is an equivalence relation. To prove it, we check three properties:

Property	Meaning
Reflexive	Every vertex is related to itself (uRu)
Symmetric	If uRv then vRu
Transitive	If uRv and vRw then uRw

Thus, mutual reachability forms **equivalence classes**.

Theorem 0.1 For all relations R , one can partition $\Omega = S_1 \uplus \dots \uplus S_t$ where S_i is a maximal equivalence set.

We can write this theorem in another way as

Theorem: Let R be an equivalence relation on the set X . Then every element $a \in X$ belongs to exactly one equivalence class and for any two equivalence classes A and A' , we either have $A = A'$ or $A \cap A' = \emptyset$.

Proof.(Optional) For a fixed $a \in X$, we define $A := \{x \in X : x \sim a\}$. We show that A is an equivalence class that contains a . Because of $a \sim a$, we have $a \in A$ and therefore $A \neq \emptyset$. For $x, y \in A$, we have $x \sim a$ and $y \sim a$ and therefore also $x \sim y$.

If $x \in A, y \in X$ and $x \sim y$, then we have $x \sim a$ and $y \sim a$ and therefore $y \in A$. Now we have shown that a is contained in at least one equivalence class.

Now we need to show that two equivalence classes A and A' are either the same or disjoint. Let $A \cap A' \neq \emptyset$ and $a \in A \cap A'$. If $x \in A$, then we have $x \sim a$ and because of $a \in A'$ we also have $x \in A'$. Therefore, we have $A \subset A'$. We can prove $A' \subset A$ analogously. Therefore, $A = A'$. □

Connectivity

In an undirected graph, two vertices u, v are connected if there exists a path between them.

A path is a sequence of vertices:

$$u = v_0, v_1, v_2, \dots, v_k = v$$

such that

$$(v_i, v_{i+1}) \in E$$

for all $0 \leq i < k$.

Connected Components

A connected component of an undirected graph is a maximal set of vertices such that every pair of vertices is reachable from each other.

Important Properties

1. Every vertex belongs to exactly one component.
2. Components form a partition of the vertex set.
3. No edge connects vertices of different components.

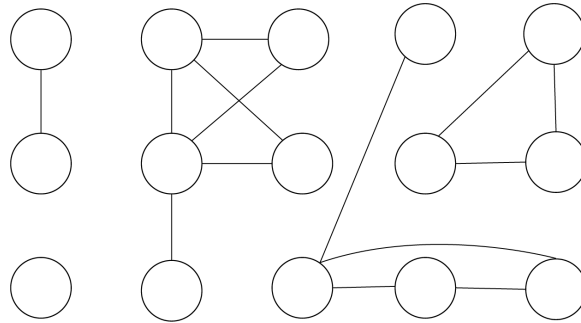


Figure 1: A Graph with 5 Connected Components

Strongly Connected Components

A Strongly Connected Component (SCC) of a directed graph is a maximal subgraph where every vertex is reachable from every other vertex within that subgraph. In other words, for any two vertices u and v in the component, there exists a directed path from u to v and also a directed path from v to u .

The diagram shows a directed graph divided into several Strongly Connected Components. Each shaded region represents a component in which every vertex is reachable from every other vertex through directed paths. The arrows indicate the direction of edges between vertices. Vertices inside the same shaded region form a strongly connected component, while edges between different regions represent connections between different components.

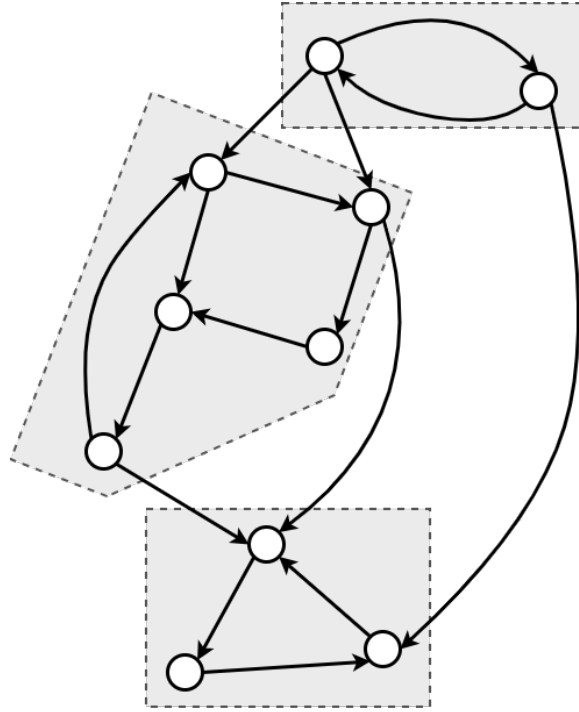


Figure 2: Strongly Connected Components (SCCs)

Properties of Strongly Connected Components

1. Every vertex of a directed graph belongs to exactly one SCC.
2. SCCs form a partition of the vertex set.
3. If each SCC is contracted into a single vertex, the resulting graph is a Directed Acyclic Graph (DAG). This graph is called the *condensation graph*.

Kosaraju's Algorithm for SCC

Steps

1. Perform DFS on the graph and record finishing times of vertices.
 - Traverse the graph using DFS.
 - After visiting all neighbors of a node, push it onto a stack.
 - This gives nodes ordered by decreasing finishing time.

2. Compute the transpose (G^R) of the graph (reverse all edges).
 - Reverse all edges in the graph.
 - If there was an edge $u \rightarrow v$, it becomes $v \rightarrow u$.
3. Process vertices in decreasing order of finishing time.
 - Initialize all vertices as unvisited again.
 - Pop a vertex from the stack.
 - If it is not visited, perform DFS on the transpose graph starting from it.
4. Each DFS traversal in the transpose graph forms one Strongly Connected Component (SCC).

Time Complexity: $O(V + E)$